

```
# =====  
# CONCEPT MAP: LEVELS OF COMPUTER VISION  
# Google Colab Code  
# =====  
  
# Install Graphviz  
!apt-get -qq install graphviz  
!pip -q install graphviz  
  
from graphviz import Digraph  
from IPython.display import Image, display  
  
# -----  
# Create Concept Map  
# -----  
dot = Digraph("ComputerVisionLevels", format="png")  
  
# Graph settings  
dot.attr(rankdir="TB")  
dot.attr(size="16,16")  
dot.attr(dpi="300")  
dot.attr(nodesep="0.5")  
dot.attr(ranksep="0.8")  
dot.attr(splines="ortho")  
  
# =====  
# MAIN NODE  
# =====
```

```
dot.node(  
    "A",  
    "LEVELS OF\nCOMPUTER VISION",  
    shape="box",  
    style="filled",  
    fillcolor="lightblue",  
    fontsize="22"  
)  
  
# =====  
# LOW-LEVEL VISION  
# =====  
  
dot.node(  
    "B",  
    "Low-Level Vision",  
    shape="box",  
    style="filled",  
    fillcolor="lightgreen"  
)  
  
dot.node("C","Image Acquisition",shape="ellipse")  
dot.node("D","Noise Removal",shape="ellipse")  
dot.node("E","Image Enhancement",shape="ellipse")  
dot.node("F","Edge Detection",shape="ellipse")  
  
# =====  
# MID-LEVEL VISION  
# =====  
  
dot.node(  
    "G",  
    "Mid-Level Vision",  
    shape="box",  
    style="filled",  
    fillcolor="lightyellow",  
    fontsize="22"  
)  
  
# =====  
# HIGH-LEVEL VISION  
# =====  
  
dot.node(  
    "H",  
    "High-Level Vision",  
    shape="box",  
    style="filled",  
    fillcolor="lightpink",  
    fontsize="22"  
)  
  
# =====  
# CONCLUSION  
# =====  
  
dot.node(  
    "I",  
    "Conclusion",  
    shape="box",  
    style="filled",  
    fillcolor="lightcyan",  
    fontsize="22"  
)  
  
# =====  
# REFERENCES  
# =====  
  
dot.node(  
    "J",  
    "References",  
    shape="box",  
    style="filled",  
    fillcolor="lightcoral",  
    fontsize="22"  
)  
  
# =====  
# APPENDIX  
# =====  
  
dot.node(  
    "K",  
    "Appendix",  
    shape="box",  
    style="filled",  
    fillcolor="lightblue",  
    fontsize="22"  
)  
  
# =====  
# END  
# =====
```

```
"G",
"Mid-Level Vision",
shape="box",
style="filled",
fillcolor="khaki"
)

dot.node("H","Segmentation",shape="ellipse")
dot.node("I","Feature Extraction",shape="ellipse")
dot.node("J","Object Representation",shape="ellipse")
dot.node("K","Shape Analysis",shape="ellipse")

# =====
# HIGH-LEVEL VISION
# =====

dot.node(
    "L",
    "High-Level Vision",
    shape="box",
    style="filled",
    fillcolor="lightskyblue"
)

dot.node("M","Object Recognition",shape="ellipse")
dot.node("N","Scene Understanding",shape="ellipse")
dot.node("O","Decision Making",shape="ellipse")
dot.node("P","Activity Recognition",shape="ellipse")

# =====
# APPLICATIONS
# =====
```

```
# =====  
  
dot.node(  
    "Q",  
    "Vision-Based Applications",  
    shape="box",  
    style="filled",  
    fillcolor="palegreen"  
)  
  
dot.node("R", "Medical Imaging", shape="box")  
dot.node("S", "Autonomous Vehicles", shape="box")  
dot.node("T", "Face Recognition", shape="box")  
dot.node("U", "Surveillance", shape="box")  
dot.node("V", "Industrial Inspection", shape="box")  
  
# =====  
# LITERATURE  
# =====  
  
dot.node(  
    "W",  
    "Literature Examples\n\n"  
    "• Gonzalez & Woods\nDigital Image Processing\n\n"  
    "• Richard Szeliski\nComputer Vision\n\n"  
    "• Forsyth & Ponce\nComputer Vision\n\n"  
    "• David Marr\nVision Theory",  
    shape="note",  
    style="filled",  
    fillcolor="lightyellow"  
)
```

```
# =====  
# RELATIONSHIPS  
# =====  
  
# Main hierarchy  
dot.edge("A","B",label="Stage 1")  
dot.edge("A","G",label="Stage 2")  
dot.edge("A","L",label="Stage 3")  
  
# Low-Level  
dot.edge("B","C")  
dot.edge("C","D",label="preprocessing")  
dot.edge("D","E")  
dot.edge("E","F")  
  
# Low -> Mid  
dot.edge("F","G",label="provides features")  
  
# Mid-Level  
dot.edge("G","H")  
dot.edge("H","I")  
dot.edge("I","J")  
dot.edge("J","K")  
  
# Mid -> High  
dot.edge("K","L",label="supports interpretation")  
  
# High-Level  
dot.edge("L","M")  
dot.edge("M","N")  
dot.edge("N","O")
```

```
dot.edge("O","P")

# Applications
dot.edge("P","Q",label="enables")

dot.edge("Q","R")
dot.edge("Q","S")
dot.edge("Q","T")
dot.edge("Q","U")
dot.edge("Q","V")

# Literature
dot.edge("W","A",label="supported by")

# =====
# SAVE & DISPLAY
# =====

filename = dot.render("Levels_of_Computer_Vision_Concept_Map")

display(Image(filename))

print("Concept Map generated successfully!")
```



Warning: Orthogonal edges do not currently handle edge labels. Try using xlabel.







